SAN PASCUAL ACADEMY

STUDENT PORTAL

Project Plan & System Design Blueprint

Purpose

A centralized online portal connecting school administration, faculty, and students for enrollment, class scheduling, quarterly grades, announcements, and student account access.

Planned technology

React + TypeScript · Laravel REST API + Sanctum · MySQL

Planning document • Version 1.0 • October 2026

Document Guide

This document consolidates the current SPA Student Portal plan: project overview, users and permissions, functional requirements, workflows, technology stack, database design, security, deployment, and phased development. It is a planning blueprint; final school policies and detailed implementation contracts must be confirmed before production development.

#	Section
1	Project Overview & Objectives
2	Users, Roles & Permissions
3	Functional Requirements
4	Core Workflows & Business Rules
5	Technology Stack & Architecture
6	Database Design Plan
7	API & Frontend Structure
8	Security, Privacy & Deployment
9	Development Phases & Acceptance Criteria
10	Open Decisions & Risks

1. Project Overview & Objectives

Project name: San Pascual Academy (SPA) Student Portal

Project type: Online, role-based school information system for Junior High School and Senior High School, covering Grades 7–12.

The portal will centralize school setup, user accounts, student enrollment, faculty-to-subject assignments, timetable proposals, quarterly grade submission and approval, and announcements. Each role receives a tailored dashboard while using the same login page.

Objectives

- Provide one online access point for Admin/Principal, Faculty, and Students.
- Reduce repetitive work by importing student lists and automatically creating student accounts.
- Keep enrollment, teaching assignments, schedules, and grades linked to the correct academic year and section.
- Use approval workflows for schedules and quarterly grades.
- Protect student data through role-based access, secure authentication, audit logs, and backups.

Initial scope

- Shared login and role-based dashboards.
- Account and profile management, school years, grade levels, sections, rooms, subjects, and enrollments.
- CSV/Excel student import, automatic account generation, and import validation.
- Faculty teaching assignments, schedule proposals, approval, and timetable viewing.
- Quarterly grades for Quarters 1–4, submission, review, correction, and finalization.
- Announcements, student profile editing, password changes, and online deployment.

Deferred from the first release unless SPA requests otherwise: attendance tracking, parent accounts, payments, messaging, a native mobile app, and automated official report cards.

2. Users, Roles & Permissions

Role	Main responsibilities	Key restrictions
Admin / Principal	Manage users, academic years, grade levels, sections, rooms, subjects, enrollments, and faculty assignments. Approve schedules, review/finalize grades, publish announcements, inspect audit logs.	Sensitive actions should be limited to authorized administrators and logged.
Faculty	View assigned classes and student lists; import/maintain permitted class lists; propose or adjust schedules; enter, submit, and correct returned quarterly grades; view announcements.	Can only manage assigned classes/subjects and authorized sections. Cannot approve their own schedule or finalize their own grades.
Student	View own profile, section, approved schedule, finalized quarterly grades, and published announcements; change password and permitted profile fields/photo.	Can access only their own records. Cannot edit grades, enrollment, or schedules.

Shared login behavior

Everyone uses the same login page. After authentication, Laravel returns the authenticated user and role; the frontend routes the user to the correct dashboard. The backend remains the authority for access decisions—hiding frontend buttons is not security.

3. Functional Requirements

Authentication & accounts

- Authenticate users through a shared login page; support logout, password change, secure password reset, and rate limiting.
- Store password hashes only. Support account status checks so deactivated accounts cannot access protected resources.
- Admin manages faculty and administrative accounts. Student accounts can be created automatically by the student-import workflow.

Admin / Principal

- Create and manage school years, grading periods, Grades 7–12, sections, rooms, and subject catalog.
- Manage faculty and student accounts; assign advisers, students to sections, and faculty to subject/section combinations.
- Import student lists with a preview, row validation, duplicate detection, and a result summary.
- Approve/reject schedule proposals and review/return/finalize grade submissions.
- Create, publish, edit, and archive announcements; inspect audit history.

Faculty

- View assigned teaching assignments and the enrolled student list for each class.
- Upload/import student lists only for sections or classes they are authorized to manage.
- Create schedule proposals, edit drafts, submit them for approval, and view decision/review comments.
- Enter grade drafts for each quarter, submit grades for review, and edit/resubmit submissions returned for correction.

Student

- Log in using a generated username and initial password; change the initial password before accessing the full dashboard.
- View own profile, current section and school year, approved schedule, finalized grades for Quarters 1–4, and published announcements.
- Edit only approved profile fields and update their profile photo and password.

4. Core Workflows & Business Rules

A. School setup and enrollment

Step	Action
1	Admin prepares the academic year, grade levels, sections, rooms, subjects, faculty accounts, and adviser/teaching assignments.
2	Faculty uploads a student list for an authorized class or section.
3	Laravel validates the file and each row, checks duplicates, and previews the import outcome.
4	For each valid new student, the system creates a student profile, login account, unique Student ID, and enrollment for the correct school year/section.
5	The system reports accounts created, existing students matched, successful enrollments, and rows requiring correction.

B. Automatic student account creation

Example student: **John Enquig**. The username pattern is **Enquig.john** (surname.firstname). The initial password is the student's generated Student ID. The exact ID must be unique and should be assigned by the system unless SPA already provides an official Student ID.

- Add a unique `username` column to `users` and a unique `student_number`/Student ID to `student_profiles`.
- Normalize names consistently; detect username collisions and generate a deterministic suffix or prompt for correction rather than creating duplicate usernames.
- If a student already exists, match by a reliable unique identifier and add/update the enrollment only according to approved rules; do not create a duplicate account.
- Store only a secure password hash. Require a password change on first login using a `must_change_password` flag or equivalent account state.
- Do not export or display a bulk list of passwords. Give initial credentials to students through a controlled school process.
- If a faculty upload creates accounts, the faculty role must be restricted to the assigned section(s); admin retains school-wide user management.

C. Schedule workflow

- Faculty edits a draft schedule proposal and submits it to the admin.
- The system checks overlaps involving teacher, room, and section. Admin approval rechecks conflicts before activation.
- Editing an approved timetable creates a new proposal/version; the current approved timetable remains active until the replacement is approved.
- Students see only approved/active schedules.

D. Quarterly grade workflow

- Faculty records grades for students in assigned classes and saves a draft.
- Faculty submits a grade batch for one teaching assignment and grading period.
- Admin either returns the batch with comments or finalizes it. Returned grades can be corrected and resubmitted.
- Students see only finalized grades. Post-finalization corrections require authorized reopening/correction and an audit record.
- Record the old value, new value, actor, timestamp, and reason for sensitive grade corrections.

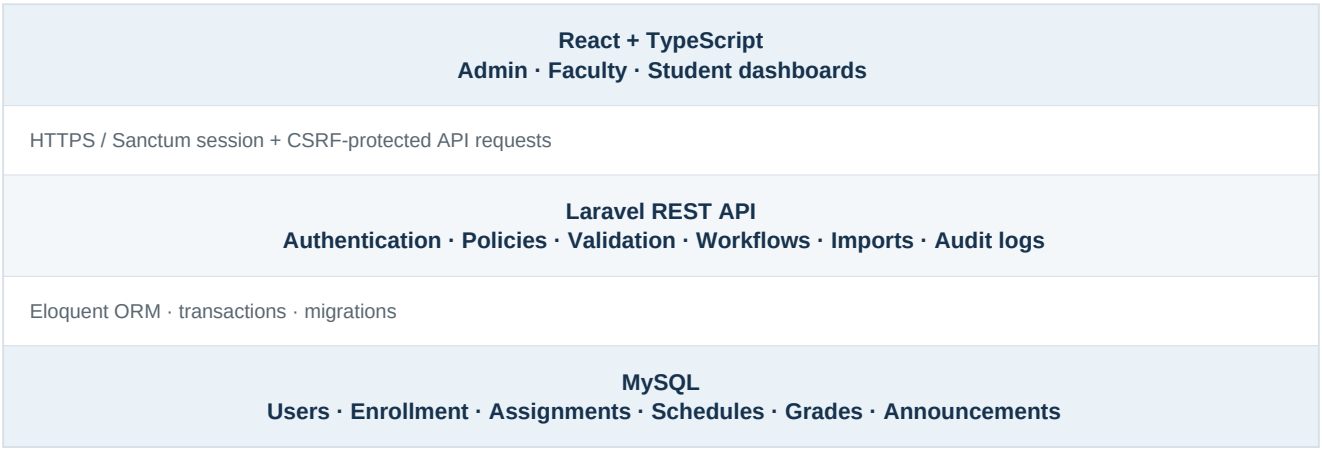
Cross-cutting rules

- Enrollments are tied to a school year and section. Historical enrollment and grade records must remain available according to school retention rules.
- A student's grade entry must match the student's enrollment, the relevant teaching assignment, and the grading period/school year.
- Use database constraints where practical and backend validation/transactions for rules that span related records.

5. Technology Stack & Architecture

Layer	Technology	Purpose
Frontend	React + TypeScript + Vite	Build dashboards and reusable UI components.
Routing	React Router	Navigation and route-level user experience.
Styling	Tailwind CSS or standard CSS	Responsive layouts and consistent styling.
API client	Axios or fetch	Communicate with the Laravel API.
Forms	React Hook Form + Zod (optional)	Client-side form handling and early validation.
Backend	Laravel REST API (PHP)	Business logic, validation, permissions, imports, workflow transitions.
Authentication	Laravel Sanctum	Cookie/session authentication for a first-party React SPA.
Authorization	Laravel policies and middleware	Enforce role and record-level permissions.
ORM & schema	Eloquent + Laravel migrations	Relationships, schema versioning, and database access.
Database	MySQL	Structured academic and account records.
Spreadsheet import	Laravel Excel or a maintained CSV/XLSX library	Read and validate uploaded student lists.
Testing	PHPUnit/Pest, frontend tests, Postman	Test API rules, permissions, imports, and workflows.
Version control	Git + GitHub	Source history, collaboration, and code review.
Hosting	Vercel or static host + Laravel-compatible PHP host + managed MySQL	Online access with separately managed frontend, backend, and database.

Architecture



Use one backend for the first release; Laravel and Express.js do not need to be combined. Keep frontend and backend as separate deployable components, with the API enforcing every permission.

6. Database Design Plan

The database should be relational, normalized, and managed through Laravel migrations. The following is the proposed core schema; detailed column types, foreign-key actions, and indexes are to be finalized in implementation planning.

A. Accounts and profiles

Table	Main fields / purpose
users	id (PK), name, username (unique), email (nullable/unique if used), password hash, role, status, must_change_password, email_verified_at, timestamps
student_profiles	id (PK), user_id (unique FK), student_number (unique), birth_date (if required), contact_number, profile_photo_path
faculty_profiles	id (PK), user_id (unique FK), employee_number (unique), contact_number, profile_photo_path

B. Academic structure

Table	Main fields / purpose
school_years	id, name, start_date, end_date, status; identify the academic year.
grading_periods	id, school_year_id (FK), quarter_number (1–4), start_date, end_date, status; unique school_year_id + quarter_number.
grade_levels	id, name, level_number; Grades 7–12.
rooms	id, room_name, capacity, status.
sections	id, school_year_id, grade_level_id, name, adviser_faculty_id (nullable FK), room_id (nullable FK).
subjects	id, subject_code, subject_name, grade_level_id (nullable FK), status.

C. Enrollment and teaching

Table	Main fields / purpose
enrollments	id, student_profile_id (FK), school_year_id (FK), section_id (FK), status, enrolled_at; unique student + school year if one enrollment per year is the rule.
teaching_assignments	id, faculty_profile_id (FK), subject_id (FK), section_id (FK), school_year_id (FK), status; unique combination unless co-teaching rules require otherwise.

D. Schedule workflow

Table	Main fields / purpose
schedule_proposals	id, teaching_assignment_id (FK), version_number, status, submitted_at, reviewed_by (FK to users), reviewed_at, review_comment.
schedule_entries	id, schedule_proposal_id (FK), room_id (FK), day_of_week, start_time, end_time; validate start_time < end_time.

Recommended proposal states: draft, pending, approved, rejected, superseded. Maintain an approved version until a replacement is approved.

E. Quarterly grades

Table	Main fields / purpose
grade_submissions	id, teaching_assignment_id (FK), grading_period_id (FK), status, submitted_by, submitted_at, reviewed_by, reviewed_at, review_comment.
grade_entries	id, grade_submission_id (FK), enrollment_id (FK), grade_value, remarks, updated_by, timestamps.

Recommended states: draft, submitted, returned, finalized. Enforce one active submission per assignment and grading period, with a deliberate revision strategy. Enforce one entry per enrollment within a submission. The grading scale and

calculation rules remain subject to SPA policy.

F. Announcements, imports, audit

Table	Main fields / purpose
announcements	id, title, body, created_by, published_at, expires_at, status.
announcement_audiences	id, announcement_id, audience_type, grade_level_id (nullable), section_id (nullable); supports targeted announcements.
import_batches	id, uploaded_by, original_filename, status, total_rows, success_count, error_count.
audit_logs	id, user_id, action, entity_type, entity_id, old_values, new_values, reason, created_at. Never store passwords or tokens.

7. API & Frontend Structure

Proposed API route groups

Module	Example routes
Authentication	POST /api/login · POST /api/logout · GET /api/me
Users	GET/POST /api/users · PATCH /api/users/{id} · POST /api/users/{id}/reset-password
School setup	/api/school-years · /api/grading-periods · /api/grade-levels · /api/sections · /api/rooms · /api/subjects
Enrollment	POST /api/enrollments/import · GET /api/enrollments · PATCH /api/enrollments/{id}
Teaching assignments	/api/teaching-assignments
Schedules	/api/schedule-proposals · /api/schedule-proposals/{id}/submit · /api/schedule-proposals/{id}/review
Grades	/api/grade-submissions · /api/grade-submissions/{id}/submit · /api/grade-submissions/{id}/review
Student portal	GET /api/student/schedule · GET /api/student/grades · GET /api/student/profile
Announcements	GET /api/announcements · POST /api/announcements · PATCH /api/announcements/{id}
Profile	PATCH /api/profile · POST /api/profile/photo · POST /api/password/change

These are route proposals, not a final API contract. Every route needs request/response definitions, validation, authorization, and error behavior. Review endpoints should accept explicit actions such as `finalize` or `return`, rather than arbitrary status values.

Suggested frontend page groups

Shared	Admin	Faculty	Student
Login, password reset/change, profile, announcements	Overview; users; school years; grade levels; sections/rooms; subjects; imports/enrollment; faculty assignments; schedule approvals; grade approvals; audit logs	My classes; class lists; my schedule; schedule proposals; quarterly grades; submission status	Overview; my schedule; my grades; announcements; my profile

8. Security, Privacy & Deployment

- Use HTTPS in production and secure, HTTP-only cookies with appropriate SameSite settings for session authentication.
- Configure Sanctum stateful domains, CORS, and CSRF protection for the actual frontend and API domains.
- Enforce authorization in Laravel policies/middleware on every request, including record-level ownership and assigned-class checks.
- Use password hashing, login rate limits, secure password reset, and mandatory first-login password change for generated student credentials.
- Validate uploaded spreadsheet type/size/content; protect against malformed rows and duplicate records; process imports transactionally where practical.
- Store uploads safely; validate profile images and prevent private student data from being served through public paths.
- Use least-privilege database accounts, environment variables for secrets, regular encrypted backups, and a tested restoration process.
- Keep audit records for sensitive changes; do not log credentials, session cookies, or authentication tokens.
- Apply appropriate privacy controls and retention practices for minors' data and comply with applicable Philippine data-protection requirements.
- Confirm the selected hosting provider supports the required PHP/Laravel version, persistent storage, background jobs if needed, scheduled tasks, and MySQL connectivity.

9. Development Phases & Acceptance Criteria

Phase	Deliverables	Completion criteria
1. Foundation	React/Vite, Laravel API, MySQL, migrations, local environment, Git repository.	Migrations run reliably; core relationships and development setup are documented.
2. Authentication	Shared login, logout, role routing, profile endpoints, account status, policies.	Each role can access only authorized endpoints; deactivated accounts are blocked.
3. Admin setup	School years, grading periods, grade levels, sections, rooms, subjects, user management.	Admin can prepare a school year through the UI without manual database edits.
4. Enrollment & assignments	Student import, validation, automatic accounts, section enrollment, advisers, teaching assignments.	Import preview works; duplicates are detected; valid students get the right account and enrollment.
5. Schedule workflow	Draft proposals, overlap checks, approval/rejection, versioning, student timetable.	Unapproved schedules never replace the active approved timetable.
6. Grade workflow	Draft grade batches, submission, return, correction, finalization, audit trail.	Students see only finalized grades; unauthorized changes are rejected and audited.
7. Student portal	Schedule, quarterly grades, announcements, profile/photo, password change.	A student can view only their own authorized records.
8. Testing & deployment	Permission tests, import tests, schedule/grade tests, production configuration, backups, documentation.	Acceptance tests pass and a backup restoration is demonstrated.

Recommended testing scenarios

- Attempt to access another student's grades by changing a URL or API ID; access must be denied.
- Upload a file with duplicate student IDs, duplicate names, malformed rows, or username collisions; the system must report the outcome without corrupting records.
- Submit conflicting schedule proposals; the system must block conflicting approval.
- Attempt to edit a submitted/finalized grade without authorization; the request must be rejected.
- Change a student's section in a later school year; prior-year grades must remain attached to the historical enrollment.
- Deactivate an account and verify that new protected requests are denied.

10. Open Decisions & Risks

The following policies must be confirmed with SPA rather than assumed during implementation.

Decision	Why it matters	Current status
Official grading policy	Grade range, transmutation, rounding, and final-grade formula must match school policy.	Unresolved; confirm with SPA.
Student ID source	The Student ID is the initial password; decide whether SPA already issues IDs or the system generates them.	Unresolved; system-generated unless SPA has official IDs.
Username collision rule	Two students can share the same surname and first name.	Use deterministic suffixing or admin resolution; finalize format.
Student identity matching	Name alone is not reliable for matching imported students.	Prefer official Student ID or another school-approved unique identifier.
Schedule format	Determine whether SPA uses a free weekly timetable, fixed periods, or both.	Unresolved; confirm with school.
Adviser rules	Determine whether one or multiple advisers may be assigned per section.	Provisional: one adviser per section; faculty may teach multiple sections.
Import formats	Choose initial supported file formats and required columns.	Provisional: Excel (.xlsx) and CSV.
Announcement audiences	Determine whether announcements can target roles, grades, or sections.	Proposed: school-wide and optional targeted audiences.
Grade corrections	Define who may reopen finalized grades and whether extra approval is needed.	Admin-controlled, audited corrections recommended.
Retention and privacy	Determine how long to keep student records, logs, and uploads.	Must align with school policy and applicable law.

Key risks and mitigations

- **Duplicate accounts:** use stable identifiers, unique constraints, import previews, and explicit matching rules.
- **Temporary credential exposure:** hash the initial password, require a change at first login, and avoid bulk password exports.
- **Incorrect grades or schedules:** use explicit workflow states, review gates, transactional updates, and audit logs.
- **Cross-student data exposure:** enforce record-level authorization in Laravel and test negative access cases.
- **Data loss:** schedule backups and regularly test restoring them.
- **Policy mismatch:** confirm grading, ID, enrollment, schedule, and retention policies with SPA before final implementation.

Recommended next step

Confirm the open school policies, then produce the implementation-ready schema specification with exact MySQL column types, primary/foreign keys, indexes, unique constraints, delete/update behavior, and a prioritized task breakdown. Implement in phases, beginning with database foundations and authentication.

End of planning document. This plan reflects the currently agreed scope and should be updated as SPA confirms school policies and requirements.